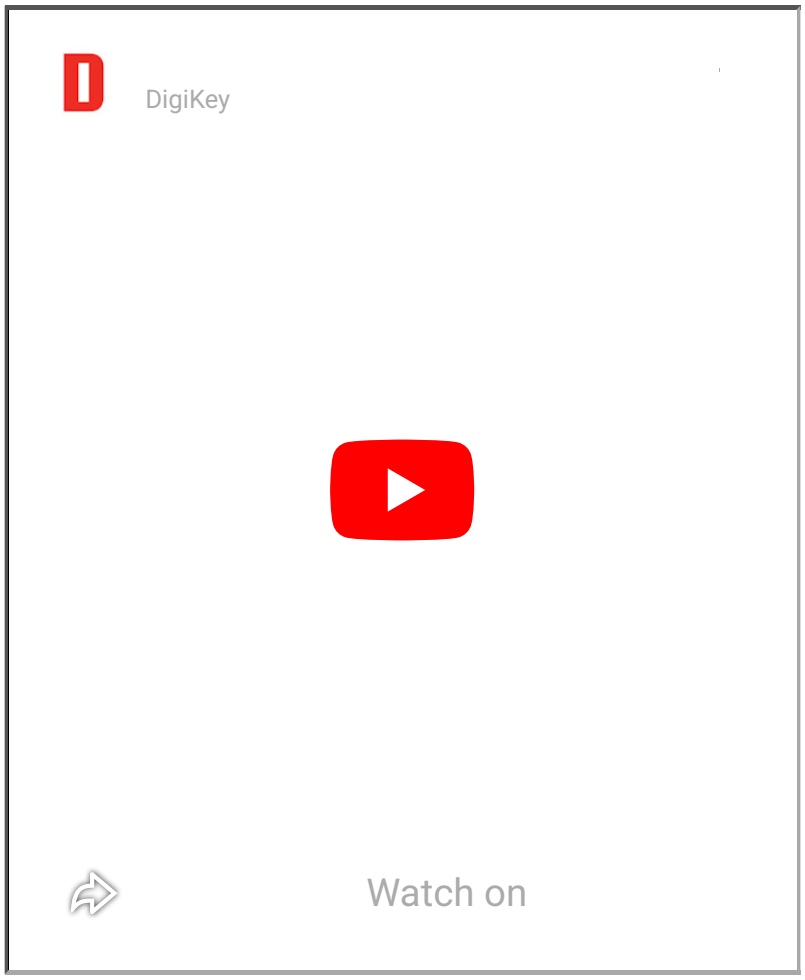
DigiKey Maker.io

Login Create Maker Profile



Related Content



What is Machine Learning?
By Maker.io Staff



Understanding the Basic Machine Learning Terminology
By Maker.io Staff



How does a CoreXY Gantry System Work?
By Andrew Kazour

Intro to Reinforcement Learning Using Gymnasium and Stable Baselines3

886 3

2023-07-17 | By ShawnHymel

License: Attribution

Reinforcement learning (RL) is a form of machine learning that involves artificial intelligence (AI) agents learning to interact with an environment in order to maximize cumulative rewards. It often involves a good amount of trial and error on the part of the agent.

To learn more about the fundamentals of RL, see the following video:

The video ends with a challenge to solve the inverted pendulum problem using reinforcement learning. This tutorial acts as a solution to that problem: we'll review the basics of reinforcement learning and then present code that solves the gymnasium *pendulum* environment. Note that in machine learning, there is often more than one way to tackle a problem, so your answer might be different!

Overview of Reinforcement Learning

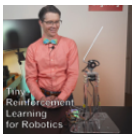
Reinforcement learning started with the work of Richard Bellman in the 1950s based on research by Hamilton and Jacobi from the 1800s. RL involves finding an optimal solution to general control theory



How to Create Custom ML Models with Google MediaPipe Model Maker
By Maker.io Staff



How To Prevent Overfitting in Machine Learning
By Maker.io Staff



Tiny Reinforcement Learning (TinyRL) for Robotics
By ShawnHymel

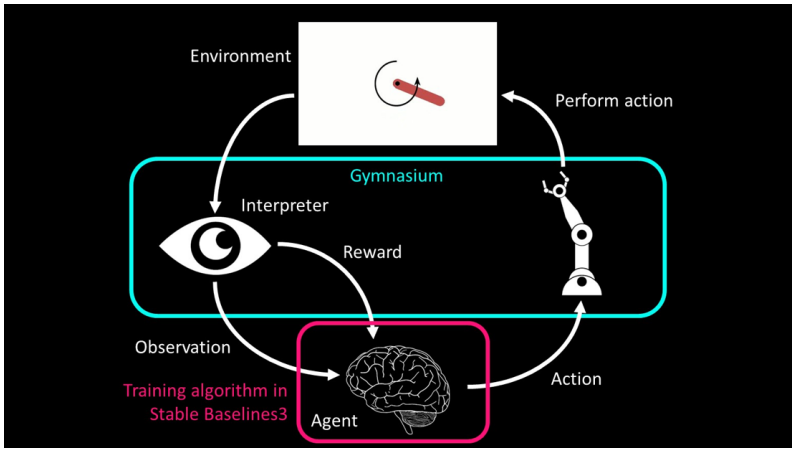


Teach an AI to play QWOP
By ShawnHymel



Hyperparameter Optimization with Meta Ax
By ShawnHymel

problems that can solve a wide range of tasks. An *agent* is the AI decision-making process that takes in observations, chooses actions, and learns from rewards.



The *environment* is the world we intend to interact with. This can be a board game, a video game, a virtual environment, or the real world. We use wrapper code to sense the environment to generate an *observation* as well as perform the actions as decided by the agent. Our program should also be able to assign one or more *rewards*. Note that these rewards are still considered part of the environment (not the AI!), and it is our job as programmers to design these rewards to assist with the training process.

In our program, we will use the Farama Foundation Gymnasium (gym) Python package to wrap the environment, send observations and rewards to the AI agent, and perform the actions requested by the agent. The DLR-RM Stable Baselines3 (SB3) package contains a number of popular, modern RL algorithms that we will use to train the agent.

During training, the agent chooses actions (either randomly or based on a policy). After this action is performed, the environment provides a new observation and a reward, which is fed back into the training algorithm. The algorithm updates parameters in the agent to (ideally) choose better actions in the future that generate higher estimated future total rewards.



How to Tune a PID Controller
By ShawnHymel



Intro to TinyML Part 1: Training a Model for Arduino in TensorFlow
By ShawnHymel

View All Projects and Tutorials

Note that the term “reward” is used to indicate the numerical reward value generated from a single step (single action and subsequent observation/reward) whereas the term “return” refers to the total discounted future rewards (from future time steps) added to the current reward. I highly recommend watching the video above if you’d like to learn more about some of the math behind RL.

Inverted Pendulum Problem

The inverted pendulum is a classic control theory problem. The idea is to swing a rotating pendulum from any position to the “up” position and hold it there. The assumption is that the force you apply to the pendulum must be less than the force required to simply move the pendulum up; you must rely on multiple swings to get the pendulum in the upright position.

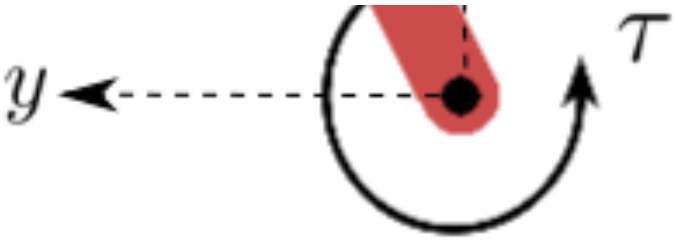
We will use the pendulum environment built into gymnasium to make setup easier. The pendulum starts in a random position, and your agent must apply forces to swing the pendulum into the upright position and hold it there. The following diagram (source: https://gymnasium.farama.org/environments/classic_control/pendulum/) specifies the coordinate system for the pendulum:



Shipping update

Due to increased fuel surcharges, rates for some U.S. domestic shipping methods have increased.

We are committed to securing the best possible rates and will continue working to keep these costs as low as possible. View new rates



- (x, y) - Cartesian coordinates of the pendulum's end [meters]
- θ - Angle (counterclockwise) of the pendulum from the upright position [radians]
- τ - Torque (counterclockwise) applied to the pendulum [Nm]

The action space is a single continuous value between -2.0 and 2.0, which constitutes the torque applied to the pendulum in newton-meters.

The observation space is 3 continuous values:

- x position of the pendulum's free end where x = cos(θ)
- y position of the pendulum's free end where y = sin(θ)
- Angular velocity of the pendulum

The reward function is given as a function of the pendulum's angle (theta), angular velocity (theta_dt), and torque:

$r = -(theta^2 + 0.1 * theta_dt^2 + 0.001 * torque^2)$

Note the negative (-) sign: this is a *penalty*, as the best reward the agent can achieve is 0. To maximize the reward, the agent should try to minimize *theta* (have the pendulum in the upright position), minimize *theta_dt* (pendulum not moving), and minimize *torque* (use as little force as possible to keep the pendulum there).

The pendulum will start at a random angle between [-π, π] and at a random angular velocity between [-1, 1].

Your job is to create and train an agent that will balance the pendulum upright using as little torque as possible. I highly recommend trying this on your own before looking at the solution below. You are welcome to use the cartpole example from the video as a starting point (the code for the cartpole solution can be found here).

Choosing an Algorithm and Hyperparameters

One of the most difficult parts of reinforcement learning (other than wrapping your head around the underlying math) is choosing an algorithm and tuning the hyperparameters. Dozens of RL algorithms exist to solve a variety of control and decision-making problems. Each has their own advantages and limitations.

Stable Baselines 3 is a collection of RL algorithm implementations, and it is frequently used in many research efforts. The following table is an overview of the current (at the time of writing) algorithms available in SB3. They reflect the state-of-the-art and industry RL algorithms most commonly used (as of July, 2023).

Algorithm	Acronym	Model	Policy	Action Space	Observation Space
Advantage Actor Critic	A2C	Model-free	On-policy	Continuous or discrete	Continuous or discrete
Deep Deterministic Policy Gradient	DDPG	Model-free	Off-policy	Continuous	Continuous or discrete
Deep Q Network	DQN	Model-free	Off-policy	Discrete	Continuous or discrete
Proximal Policy Optimization	PPO	Model-free	On-policy	Continuous or discrete	Continuous or discrete
Soft Actor Critic	SAC	Model-free	Off-policy	Continuous	Continuous or discrete
Twin Delayed Deep Deterministic Policy Gradient	TD3	Model-free	Off-policy	Continuous	Continuous or discrete

The action space for our pendulum environment is continuous. As a result, Deep Q Networks (DQN) will not work for us. Proximal Policy Optimization (PPO) is one of the newer algorithms and is quite popular for solving a range of control problems. So, we'll go with that. Feel free to try the others, though!

RL is notorious for being sensitive to hyperparameters, which means that slight changes in your settings can mean the difference between a working agent and one that does nothing. Training (for more complex problems) can take a long time (hours or days). As a result, researchers often rely on powerful computer clusters with AI acceleration (e.g. GPUs) to train multiple agents in parallel to find a possible solution. Luckily, training an agent for our simple pendulum problem should only take a few minutes.

Also, RL training can be extremely random, which might mean recreating your agent (with a new set of randomly initialized parameters) and training again to see if it worked. Setting a random seed to a particular value is often considered another hyperparameter to ensure reproducibility in an agent. For our purposes, simply recreate your agent ("model") and retrain if you think you have your hyperparameters dialed in.

You can read more about the hyperparameters for PPO here.

When learning RL, it can be frustrating to find

something that works to see early successes. As a result, I highly recommend looking up hyperparameters from a known good source and then tweaking from there. DLR-RM maintains a collection of trained agents for various common environments in gymnasium, and they publish their hyperparameters in a GitHub repository. For training PPO on pendulum-v1, we will use the hyperparameters found here.

Training and Testing the Agent

All of the code to train and test our agent using PPO on the pendulum problem can be found here. If you click the "Open in Colab" button at the top of the code, you can run the notebook in Google Colab without needing to install anything locally.

To start, we install specific versions of Gymnasium and Stable Baselines3 to ensure that they will work together properly.

Copy Code

```
!python -m pip install gymnasium==0.2
!python -m pip install stable-baselir
```

Next, we import the required packages.

Copy Code

```
import gymnasium as gym
import stable_baselines3 as sb3
import matplotlib.pyplot as plt
import cv2
import numpy as np

# Check versions
print(f"gym version: {gym.__version__
print(f"cv2 version: {cv2.__version__
```

We then create the pendulum environment built into Gymnasium.

Copy Code

```
# Create the environment
env = gym.make('Pendulum-v1', render_
```

We create a function to test the given agent ("model") in our environment. Note that this resets the environment and runs forever until terminated or truncated comes back False. Our model is used to predict an action from a given observation (known as the *policy*). This action is used to take a step (with the .step() function) in the environment, which returns a new observation, reward, and whether or not the environment has terminated/truncated.

If a video handle (from OpenCV) is passed in, each step is rendered and added to the video. If a message (msg) is passed in, that text will appear on the top-left of the video.

Copy Code

```
# Function that tests the model in th
def test_model(env, model, video=None

    # Reset environment
    obs, info = env.reset()
    frame = env.render()
    ep_len = 0
    ep_rew = 0

    # Run episode until complete
    while True:

        # Provide observation to poli
        action, _ = model.predict(obs

        # Perform action, update tota
        obs, reward, terminated, trur
        ep_rew += reward

        # Record frame to video
```

```
        if video:
            frame = env.render()
            frame = cv2.cvtColor(fram
            frame = cv2.putText(
                frame,
                msg,
                (10. 25).
```

From there, we create a dummy agent that simply selects actions randomly.

Copy Code

```
# Model that just predicts random act
class DummyModel():

    # Save environment
    def __init__(self, env):
        self.env = env

    # Always output random action reg
    def predict(self, obs):
        action = self.env.action_spac
        return action, None
```

We then configure our video writer object and run a few episodes with our random agent. Feel free to download the output video (1-random.mp4) to see how this agent performs (probably poorly).

Copy Code

```
# Recorder settings
FPS = 30
FOURCC = cv2.VideoWriter.fourcc('m',
VIDEO_FILENAME = "1-random.mp4"

# Use frame from environment to compu
width = frame.shape[1]
height = frame.shape[0]

# Create recorder
video = cv2.VideoWriter(VIDEO_FILENAM
```

```
# Try running a few episodes with the
dummy_model = DummyModel(env)
for ep in range(5):
    ep_len, ep_rew = test_model(env,
    print(f"Episode {ep} | length: {e

# Close the video writer
video.release()
```

Next, we initialize our model (agent) to train with the PPO algorithm and set some hyperparameters. As noted, our hyperparameters come from the rl-baseline3-zoo repository.

Copy Code

```
# Initialize model
model = sb3.PPO(
    'MlpPolicy',
    env,
    learning_rate=0.001,       # Lear
    n_steps=1024,               # Num
    batch_size=64,              # Mir
    gamma=0.9,                 # Disc
    ent_coef=0.0,               # Ent
    use_sde=True,               # Use
    sde_sample_freq=4,          # Num
    policy_kwargs={'net_arch': [64, 6
    verbose=0                   # Pri
```

With our model configured, we train. To make our demo more visually appealing, we divide the training into “rounds.” In each round, we train for a given number of steps and then test the model in our environment 100 times. The first test is recorded to video, and the episode lengths and rewards are averaged over all 100 tests.

This can take 5-10 minutes. Once done, you can download the output video (2-training.mp4) to see the training progress.

Copy Code

```
# Training and testing hyperparameter
NUM_ROUNDS = 20
NUM_TRAINING_STEPS_PER_ROUND = 5000
NUM_TESTS_PER_ROUND = 100
MODEL_FILENAME_BASE = "pendulum-ppo"
VIDEO_FILENAME = "2-training.mp4"

# Create recorder
video = cv2.VideoWriter(VIDEO_FILENAM

# Train and test the model for a numb
avg_ep_lens = []
avg_ep_rews = []
for rnd in range(NUM_ROUNDS):

    # Train the model
    model.learn(total_timesteps=NUM_T

    # Save the model
    model.save(f"{MODEL_FILENAME_BASE

    # Test the model in several episc
    avg_ep_len = 0
    avg_ep_rew = 0
    for ep in range(NUM_TESTS_PER_ROU

        # Only record the first test
```

To visualize how our agent progressed over time, we can plot the average test lengths and rewards for each round.

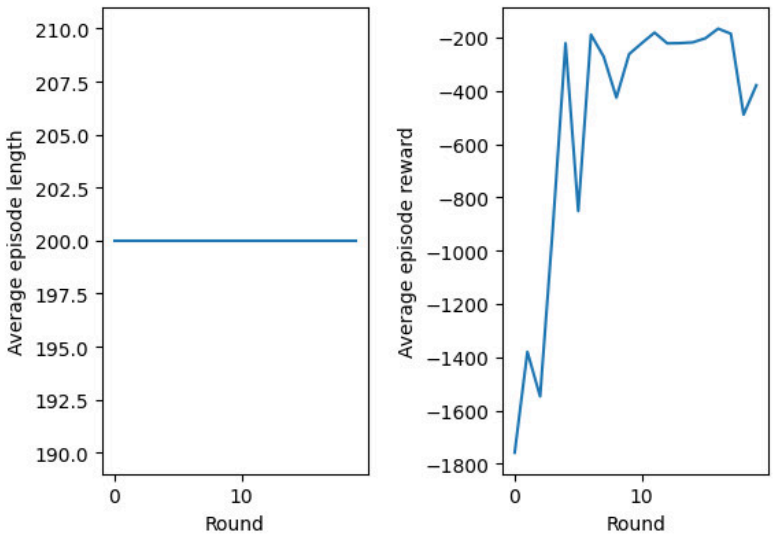
Copy Code

```
# Plot average test episode lengths a
fig, axs = plt.subplots(1, 2)
fig.tight_layout(pad=4.0)
axs[0].plot(avg_ep_lens)
axs[0].set_ylabel("Average episode le
axs[0].set_xlabel("Round")
axs[1].plot(avg_ep_rews)
```

```
axs[1].set_ylabel("Average episode re
axs[1].set_xlabel("Round")
```

This should give you something similar to the following plots. Note that agents can sometimes forget what they've learned and suddenly start performing worse! As a result, it's usually best to choose the agent with the highest average rewards. We save the model every round, so you can just choose the model file that provided the highest average reward.

If your agent does not provide acceptable average rewards, try running the previous 3 cells (recreate the model, retrain the model, and plot the results) to see if you get better results.



Choose a model that provided the best average rewards, load it, and call our test_model() function using that model. For example, in the plot above, we can see that the model from round 17 gave the best average test results. So, change the MODEL_FILENAME value to "pendulum-ppo_17" and run the following cell.

Copy Code

```
# Model and video settings
MODEL_FILENAME = "pendulum-ppo_17"
```

```
VIDEO_FILENAME = "3-testing.mp4"

# Load the model
model = sb3.PPO.load(MODEL_FILENAME)

# Create recorder
video = cv2.VideoWriter(VIDEO_FILENAM

# Test the model
ep_len, ep_rew = test_model(env, mode
print(f"Episode length: {ep_len}, rew

# Close the video writer
video.release()
```

That will test the agent in the environment for one episode. The video will be saved as “3-testing.mp4.” Feel free to download it and give it a watch. With some luck, your pendulum should be balancing almost perfectly upright!



When you are done, don’t forget to close your environment.

Copy Code

```
# We're done with the environment
env.close()
```

Hopefully, this helps you get started using Gymnasium and Stable Baselines3 to build and train your own AI agents!

Recommended Reading

Reinforcement learning is a deep and complex field. The following resources should hopefully help if you'd like to dive deeper into the subject.

Full courses:

- Coursera: Reinforcement Learning Specialization
- EdX: Fundamentals of Deep Reinforcement Learning
- Udemy: Artificial Intelligence: Reinforcement Learning in Python
- OpenAI: Spinning Up in Deep RL
- Hugging Face: Deep RL Course
- Google DeepMind: Introduction to Reinforcement Learning with David Silver

Videos:

- Reinforcement Learning in 3 Hours | Full Course using Python by Nicholas Renotte
- Deep RL for Robotics talk by Mat Kelcey

Books:

- Reinforcement Learning: An Introduction by Sutton and Barto
- Grokking Deep Reinforcement Learning by

Miguel Morales

Articles:

- Reinforcement Learning Algorithms — an intuitive overview
- Which Reinforcement learning-RL algorithm to use where, when and in what scenario?
- Q-Learning vs. Deep Q-Learning vs. Deep Q-Network
- Deep Q Networks (DQN) With the Cartpole Environment
- RL — Proximal Policy Optimization (PPO) Explained
- Proximal Policy Optimization (PPO)

Have questions or comments? Continue the conversation on TechForum, DigiKey's online community and technical resource.

Start Discussion 0 replies

TechForum